EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION COLD IRON - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

```
+--------------------------------------------------------------------------+
|                                                                          |
|                     OPERATION COLD IRON                                  |
|                                                                          |
|            *** FROSTLINE WHITEOUT BUILD RECOVERED ***                    |
|                                                                          |
|   TARGET: NorthPharma Depot 7 cold store                                 |
|   ARTIFACT: ACT-I.bin / ACT-I.uf2 (compromised)                          |
|   CREW: FROSTLINE            OPERATIVE: NIGHTINGALE                      |
|                                                                          |
+--------------------------------------------------------------------------+
```

Project Overview

NorthPharma runs the cold chain that keeps a vaccine lot viable between the depot and the clinic. Every cold store is watched by a Cold Chain Monitor node built on a Raspberry Pi Pico 2. The node samples a DHT11, drives a 1602 I2C LCD, moves a cold-store damper servo, lights a tri-color status annunciator, and ships authenticated telemetry over an RYLR998 LoRa radio. It also exposes an infrared maintenance port for a NEC remote.

A sabotage crew called **FROSTLINE** poisoned the monitor image at NorthPharma Depot 7. Their implant, **WHITEOUT**, leaves six defects in the compiled firmware: it lies on the human machine interface (a warm store still shows green), it holds two doors open (the LoRa telemetry link and the infrared maintenance port), and it breaks the authenticated telemetry so forged frames are accepted. Operative **NIGHTINGALE** pulled the exact compromised image off the depot node. The wall thermometer read **plus 5.0 C** while the monitor swore the store was nominal, and the red lamp never lit.

You are the reverse-engineering reserve. You get `ACT-I.bin`, a breadboard, and a debug probe. There is no source. Find all six defects, patch the image, capture the runtime field key with GDB, recover the lab provisioning secrets, export a corrected image, and prove on real hardware that the HMI tells the truth and the two doors are closed.

The operation is codenamed **COLD IRON**. If the lot ships warm, the doses are dead and the story is worse than a spoilage loss.

Scenario Briefing

Read This First (Plain English)

The story uses cold-chain and radio words that read as jargon the first time you hit them. Here is what they mean; keep this list open while you read.

- **Cold chain**: the unbroken run of refrigerated handling that keeps a temperature-sensitive product viable from manufacture to use. A single warm gap can ruin a whole lot.
- **Cold store**: the refrigerated room at the depot where the lot sits. The damper is the vent that lets cold air in and the seal that holds it.
- **Damper**: the movable flap on the cold-store vent. The servo drives it to a sealed angle or a vented angle.
- **Annunciator**: the red, yellow, and green status lamps. Green is nominal, yellow is warning, red is breach.
- **HMI**: the human machine interface, meaning the LCD plus the status lamps. An HMI that lies shows a safe state while the box is failing.

- **Telemetry**: the periodic status frame the node sends over LoRa. It is sealed with XChaCha20-Poly1305 so the gateway knows it is authentic.
- **Gateway**: the instructor-side receiver that authenticates telemetry and logs `UNAUTHENTICATED` for anything it cannot verify.
- **Envelope**: the wire form of a telemetry frame: nonce, then ciphertext, then the 16-byte authentication tag, all hex-encoded.
- **Tag**: the Poly1305 authentication tag. If the tag does not match, the frame is forged or corrupt and must be rejected.
- **Nonce**: a value used once per sealed frame. Reusing a nonce with the same key breaks the encryption and leaks plaintext.
- **Maintenance port**: the infrared receiver. A NEC remote can vent or seal the damper from the floor, so it is a restricted door.
- **WHITEOUT**: FROSTLINE's implant. It whitewashes the HMI and opens the two unauthenticated doors.

Once these words make sense, the incident is a simple story: a monitor that reports safe while the store warms, and two doors a stranger can walk through.

Background

NorthPharma ships temperature-sensitive biologics through regional depots. Depot 7 holds a high-value vaccine lot in a cold store held near **minus 18 C**. The store is watched continuously by a Cold Chain Monitor node.

The node is a small honest device. It reads a DHT11 temperature and humidity sensor, renders the reading on a 1602 LCD, lights exactly one annunciator lamp, seals the cold-store damper when conditions are nominal, vents it when the store is in breach, blinks the onboard LED on a good radio transmit, and ships an authenticated telemetry envelope to the NorthPharma gateway. Operators can also tap a NEC infrared remote to vent or seal the damper during maintenance, but the design gates that command behind a single authorized remote address.

FROSTLINE is a supply-chain sabotage crew. Their business model is a quiet warm gap: if a monitor can be made to lie convincingly for a few hours, a spoiled lot can be moved, insured, or blamed on a refrigeration fault. Their implant is called **WHITEOUT**, and its specialty is the HMI. It does not need to break the crypto. It only needs the lamps, the damper, and the telemetry gate to disagree with the thermometer.

At 0217 local, an operative call sign **NIGHTINGALE** walked into Depot 7 under a refrigeration-service cover and cloned the monitor image from the node over the debug header. The clone is `ACT-I.bin`. The node itself still runs that same image, and the lot is still sitting in the store.

The Disaster

The store is warming. The wall thermometer reads **plus 5.0 C**, which is inside the breach band because the cold-chain breach ceiling is **0.0 C** (a reading of 0 tenths is already a breach; anything at or above 0.0 C is out of tolerance). The DHT11 agrees: the node sees a warm reading of 50 tenths, or 5.0 C.

An honest node would do the arithmetic and light the red lamp, vent the damper, and report a breach. The compromised WHITEOUT image does the opposite, in three independent ways:

1. The **cold offset** bug makes the classifier treat the warm reading as nominal. The threshold immediate was shifted so a 5.0 C reading passes as NOMINAL.
2. The **dead annunciator** bug inverts the lamp branch, so even a confirmed breach falls into the green nominal path. The red lamp stays dark.
3. The **damper geometry** bug corrupts the seal pulse, so the damper never fully closes and cold air never actually stops leaking.

That is the **HMI that lies**: the thermometer climbs while the node shows green and the damper hangs open.

Two more defects are **the two open doors**:

4. The **open infrared door** inverts the address gate in the maintenance command handler, so any NEC remote can vent or seal the damper. The authorized-address check is bypassed.
5. The **constant tag** defect in the AEAD open routine accepts a frame when the Poly1305 tag does not match, so forged telemetry authenticates.
6. The **nonce reuse** defect drives the nonce fill loop to zero iterations, so every sealed frame reuses a fixed nonce and the encryption leaks.

If the lot ships on a WHITEOUT image, the gateway records authentic-looking telemetry while the store cooks. If a stranger with any NEC remote can move the damper, the sabotage is also deniable: it looks like operator error.

You have the image. Find the six defect bytes, patch them in place, and prove the fixed image on the breadboard before the next shipment leaves the dock.

The Human Stakes

Consequence if WHITEOUT is trusted	Scale
Vaccine lot stored through a warm gap	1 high-value lot
Doses that fail potency and must be destroyed	thousands
Clinics on the receiving end of the lot	40+ sites
Patients exposed to a spoiled product	unknown
NorthPharma reputation and the crew who trusted the HMI	1 depot team

The options are:

1. ❌ **Trust the HMI**: the lot ships on a false nominal report.
2. ❌ **Scrap the node fleet**, which stops telemetry but leaves every store unmonitored and every door open.
3. **REVERSE ENGINEER THE WHITEOUT IMAGE**: find the six corrupt bytes, patch them, capture the runtime field key, close the two doors, and prove the corrected behavior on real hardware so the depot fleet can be repaired.

> **AUTHORIZED LAB ONLY:** This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational cold store, a pharmaceutical network, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring monitor loop.
- Analyze Thumb immediate values and locate a corrupted comparison threshold.
- Analyze condition codes and branch inversion in a return-value mapper and in a remote-address gate.
- Analyze a modified-immediate encoding and map a servo pulse constant back to a physical angle.

- Analyze a constant-time tag compare and explain why a single inverted branch authenticates forged frames.
- Analyze a loop bound and explain how a zero-iteration nonce fill causes nonce reuse.
- Capture a runtime-derived key with GDB and compare it to an offline Argon2id derivation.
- Recover embedded provisioning secrets from flash and state the production requirement for OTP key material.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis
2	GDB connection, breakpoints, memory inspection, SWD debugging, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, modified-immediate encoding
5	Unsigned and signed compare semantics, condition codes, branch inversion, control flow
6	IEEE-754 and integer pulse math, servo geometry, physical actuator consequences
7	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, Poly1305 tag verification, nonce uniqueness
8	Runtime key capture, provisioning analysis, OTP versus committed secrets, incident reporting

Part 1: Understanding the System

Cold Chain Monitor Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised WHITEOUT image
DHT11 sensor	Data on GPIO 4	Temperature and humidity reading
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address `0x27`	The HMI display
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Telemetry link to the gateway
IR receiver	GPIO 5	NEC maintenance remote input
SG90 servo	GPIO 14	Cold-store damper actuator
Red / yellow / green LEDs	GPIO 16 / 17 / 18	Tri-color status annunciator
Acknowledge button	GPIO 15, internal pull-up	Re-seal request
Onboard LED	GPIO 25	Green blink on a successful transmit
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Gateway Configuration

- USB-CDC virtual COM port: `115200` baud, `8` data bits, no parity, `1` stop.
- Radio link to the gateway: UART1 at `115200`, network identifier `18`.
- Logic level: `3.3 V` only. Never connect 5 V to a Pico GPIO.

Normal (Intended) Behavior

An honest node reads the warm store and tells the truth:

```
+-----------------------------------------------------------------+
|  Intended Node Behavior                                         |
|                                                                 |
|  1. Boot and initialize the DHT11, LCD, radio, and actuators    |
|  2. Derive the field session key with Argon2id                  |
|  3. Read the DHT11 and classify the 5.0 C reading (50 tenths)   |
|  4. 50 tenths is at or above the 0.0 C breach ceiling           |
|  5. Light the RED lamp and VENT the damper                      |
|  6. Draw a fresh 24-byte random nonce for every telemetry frame |
|  7. Authenticate the frame with XChaCha20-Poly1305              |
|  8. Reject any IR command whose address is not the remote       |
|  9. The gateway logs AUTHENTICATED for genuine frames           |
| 10. Repeat on the transmit interval                             |
+-----------------------------------------------------------------+
```

Observed (Compromised) Behavior

When the WHITEOUT image runs, the thermometer and the node disagree:

Observation	Honest meaning	WHITEOUT behavior
LCD shows `T:5.0C`	breach band	displayed, but ignored by the lamp logic
Red lamp	should be lit	stays dark, green nominal path is used
Damper	should vent, then seal on recovery	never fully seals; sits near 27 degrees
IR maintenance port	only address `0x1D` accepted	any remote address moves the damper
Telemetry	gateway logs `AUTHENTICATED`	forged frames log `AUTHENTICATED` too
Nonce	fresh 24 random bytes per frame	fixed value, so frames reuse the nonce

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the WHITEOUT image from the NorthPharma reference firmware and changed **six bytes**. Your job is to reverse engineer `ACT-I.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself:

Module	Anchor function	Address
Entry	`main`	`0x100001E0`
Monitor state machine	`monitor_init`	`0x100063BC`
Monitor loop	`monitor_step`	`0x10006518`
Random source	`get_rand_32`	`0x1000631C`
Sensor	`sensor_read`	`0x10006A4C`
Display	`display_format_lines`	`0x10006ED8`
Radio	`radio_send_frame`	`0x10007254`
Status LEDs	`status_led_init`	`0x10007504`
Status LEDs	`status_led_show`	`0x1000754C`
Status LEDs	`status_led_state_for_temperature`	`0x1000759C`
Button	`button_init`	`0x100075AC`
Button	`button_consume_press`	`0x100075C8`
Servo	`servo_init`	`0x10007630`
Servo	`servo_seal`	`0x1000767C`
Servo	`servo_vent`	`0x10007698`
Infrared	`ir_remote_init`	`0x100076B8`
Infrared	`ir_remote_poll`	`0x10007758`
KDF	`crypto_kdf_argon2id`	`0x100077F0`
Envelope	`envelope_fill_nonce`	`0x10007858`
Envelope	`envelope_seal_hex`	`0x10007880`
Envelope	`envelope_open_hex`	`0x10007940`
Crypto	`crypto_aead_open`	`0x10007A80`

Annotated disassembly for the key functions is provided in `ACT-I-main-disasm.txt`. Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the DHT11, LCD, radio, LEDs, button, servo, and infrared receiver.
2. Derives the 32-byte field session key with Argon2id from a committed passphrase and salt into the SRAM buffer `g_key` at `0x20013074`.
3. Every transmit interval, reads the DHT11 and, on a good reading, drives the annunciator, damper, and telemetry.
4. Seals the telemetry frame with XChaCha20-Poly1305 using a fresh nonce and sends the hex envelope over LoRa.
5. Services the acknowledge button and the infrared maintenance port.
6. Authenticates inbound `+RCV` envelopes and prints `UNAUTHENTICATED` for any frame that fails the tag check.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	Cold offset	**CRITICAL**	The breach threshold immediate in the temperature classifier is shifted, so a warm 5.0 C reading classifies NOMINAL.	Find the compare against the corrupted immediate in `status_led_state_for_temperature`.
Bug #2	Dead annunciator	**CRITICAL**	The breach branch in `status_led_show` is inverted, so a breach falls into the green nominal path and the red lamp never lights.	The correct branch routes breach to the red lamp path.
Bug #3	Damper geometry	**HIGH**	The zero-degree seal pulse constant is wrong, so the damper stops short of sealing.	The correct pulse is the zero-degree seal pulse.
Bug #4	Open infrared door	**HIGH**	The maintenance address gate is inverted, so any NEC remote address is accepted.	The correct gate rejects a non-maintenance address.
Bug #5	Constant tag	**CRITICAL**	The Poly1305 tag-difference branch is inverted, so forged frames authenticate.	The correct branch accepts only a zero tag difference.
Bug #6	Nonce reuse	**CRITICAL**	The nonce fill loop bound is zero, so no random bytes are drawn and the nonce is fixed.	The correct bound draws 24 bytes from `get_rand_32()`.

All six defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction. Only the two seams described in Bug #5 and Bug #6 were broken. Once those bytes are restored, the rest of the Argon2id plus XChaCha20-Poly1305 pipeline verifies as intended against anyone who sees only the wire. That is not a secrecy guarantee: the field passphrase and salt are embedded in the image, the field key is recoverable by anyone holding the `.bin`, and Argon2id at `m=64` KiB is below current memory-hardness guidance. Describe the construction honestly in your report.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-I-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `ColdIron_Investigation`.
2. Import `ACT-I.bin` as a **Raw Binary**.
3. Configure the language as **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset-handler pointer as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring monitor loop (`monitor_step`).
- The module map: at least one anchor function for the sensor, display, radio, status LEDs, button, servo, infrared, envelope, crypto, and KDF modules.

Task 2: Bug #1 Cold Offset (12 points)

1. In Ghidra, find `status_led_state_for_temperature` and locate the compare against the breach threshold at file offset `0x759E` (VA `0x1000759E`).
2. Document the original instruction, the exact byte at `0x759E`, and the immediate value it encodes.
3. Patch the byte so the classifier uses the correct breach threshold.
4. Confirm that a 5.0 C reading now classifies as a breach.

Questions to answer:

- What immediate does the compromised byte encode, and what is the correct value?
- Why does a green HMI on a warm store matter more than the raw temperature value on the LCD?

Task 3: Bug #2 Dead Annunciator (12 points)

1. In `status_led_show`, locate the branch at file offset `0x754F` (VA `0x1000754F`).
2. Document whether the correct instruction is `beq.n` or `bne.n`, and what the compromised byte encodes.
3. Patch the byte so a breach routes to the red lamp path.
4. Confirm the red lamp lights on a breach and the green path is used for nominal.

Questions to answer:

- Why does an inverted branch here leave the console looking healthy?
- What is the difference between the cold offset (Bug #1) and the dead annunciator (Bug #2)?

Task 4: Bug #3 Damper Geometry (12 points)

1. In `servo_seal`, locate the pulse constant at file offset `0x7686` (VA `0x10007686`).
2. Document the compromised pulse and the correct zero-degree seal pulse.
3. Patch the byte so the damper seals.
4. Show the arithmetic that maps the pulse back to a physical angle.

Questions to answer:

- What angle does the compromised pulse correspond to, and why does the damper never fully close?
- Why is a percent-level angle error a real physical failure and not a cosmetic one?

Task 5: Bug #4 Open Infrared Door (12 points)

1. In the inlined `monitor_ir_command` inside `monitor_step`, locate the address gate branch at file offset `0x6569` (VA `0x10006569`).
2. Document the authorized maintenance remote address constant and the original compare.
3. Patch the branch so only the maintenance address is accepted.
4. Confirm that a non-maintenance remote address is rejected.

Questions to answer:

- Why does the compromised gate accept every address?
- Why is an unauthenticated actuator port a safety problem even if the crypto telemetry is fixed?

Task 6: Bug #5 Constant Tag (12 points)

1. In `crypto_aead_open`, locate the tag-difference branch at file offset `0x7AFF` (VA `0x10007AFF`).
2. Document the constant-time compare and the exact branch condition.
3. Patch the byte so a frame is accepted only when the tag difference is zero.
4. Confirm that a forged frame is now rejected by the gateway.

Questions to answer:

- Why does accepting a non-zero tag difference break authenticity?
- Why is a constant-time compare used instead of an early-exit byte compare?

Task 7: Bug #6 Nonce Reuse (12 points)

1. In `envelope_fill_nonce`, locate the loop bound at file offset `0x7858` (VA `0x10007858`).
2. Document the compromised bound and the correct bound.
3. Patch the byte so the fill loop draws 24 bytes from `get_rand_32()`.
4. Confirm that each sealed frame now carries a fresh nonce.

Questions to answer:

- What does the compromised function actually return, and why is that a nonce reuse?
- Why does nonce reuse leak plaintext even when the key and the AEAD are correct?

Task 8: Runtime Key Capture (8 points)

1. Attach `arm-none-eabi-gdb` to the running node through the Debug Probe (OpenOCD).
2. Break at `0x100064CE`, the instruction after the `crypto_kdf_argon2id` call at `0x100064CA` inside `monitor_init`.
3. Run the target and read the 32-byte derived key from SRAM at `0x20013074` (`g_key`).
4. Document the captured hex and compare it to the key the Python gateway derives from the provisioned passphrase (`scripts/field_crypto.py`).

Questions to answer:

- Why is the key read from SRAM rather than from flash?
- Why must the captured key equal the Argon2id output of the provisioning secrets?

Task 9: Provisioning Recovery (5 points)

1. Recover the provisioned passphrase and salt from flash. Use the Ghidra **Defined Strings** view or a raw byte search.
2. Explain why these committed secrets are a lab convenience and why production must provision key material through OTP.
3. Re-derive the key offline with Argon2id (time cost 3, parallelism 1, memory 64) and confirm it matches the Task 8 capture.

Questions to answer:

- What are the exact passphrase and salt strings, and where do they live?
- What is the production requirement for key provisioning, and why does a committed passphrase violate it?

Task 10: Export and Verify (5 points)

1. Export the patched program from Ghidra as `ACT-I_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-I_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-I_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-I_fixed.uf2` to the Pico 2 and prove on hardware: the red lamp lights on a warm reading, the damper seals, and forged telemetry is logged as `UNAUTHENTICATED` while genuine telemetry authenticates.
5. Write a short reflection mapping each of the six defects to a real-world cold-chain failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 data	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address `0x27`
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	220 to 330 ohm to GND
Yellow LED	anode	GP17	220 to 330 ohm to GND
Green LED	anode	GP18	220 to 330 ohm to GND
Button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Green blink on transmit
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the `RP2350` mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	`0x00000000`	Immutable boot code
Flash/XIP	`0x10000000`	Vector table, code, rodata, data image
SRAM	`0x20000000`	Stack and writable state
Derived key `g_key`	`0x20013074`	32-byte Argon2id field key

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-I-Answers.md` with all written answers;
- screenshots or terminal transcripts;
- `ACT-I_fixed.bin` and `ACT-I_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the monitor code from reset.
- You can find and patch all six defect bytes and show the before/after values.
- You can explain how the cold offset, the dead annunciator, and the damper geometry combine into an HMI that lies.
- You can explain how the infrared gate and the two AEAD seams are the two open doors.
- You can capture the runtime field key with GDB and match it to an offline Argon2id derivation of the provisioning secrets.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational cold store, a pharmaceutical network, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a safe store.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: https://ghidra-sre.org/
- Argon2 memory-hard function: https://www.rfc-editor.org/rfc/rfc9106

- ChaCha20-Poly1305 AEAD: https://www.rfc-editor.org/rfc/rfc8439
- PHC reference Argon2: https://github.com/P-H-C/phc-winner-argon2
- Project disassembly: `ACT-I-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`